

Universidad del Valle de Guatemala

Facultad de Ingeniería

Departamento de Computación

Redes

1966

UNIVERSIDAD

GUATEMALA



LABORATORIO II

ESQUEMAS DE DETECCIÓN Y CORRECCIÓN DE ERRORES

André Emilio Pivaral López – 23574

Excelencia que trasciende

Catedrático: Kevin Antonio Velásquez Aguilar

Sección: 10

Nueva Guatemala de la Asunción, 2 de agosto de 2026

Índice

1. Resumen	1
2. Descripción del Laboratorio	1
2.1. Planteamiento	1
2.2. Objetivos	2
2.3. Arquitectura implementada	2
2.4. Algoritmos implementados	3
2.4.1. Código de Hamming	3
2.4.2. Fletcher checksum	3
2.5. Herramientas y organización del código	4
3. Metodología experimental	4
3.1. Mensajes de prueba	4
3.2. Escenarios de alteración	5
3.3. Simulación estadística	5
4. Resultados	6
4.1. Compilación y puesta en marcha	6
4.2. Generación de tramas y overhead	6
4.3. Escenario 1: cero errores	8
4.4. Escenario 2: un error	10
4.5. Escenario 3: dos o más errores	13
4.6. Módulo de ruido probabilístico	17
4.7. Transmisión mediante sockets	20
4.8. Tamaño de bloque configurable	21
4.9. Intento de engañar a los algoritmos	22
4.9.1. Caso 1: síndrome nulo en Hamming	22

4.9.2. Caso 2: corrección equivocada en Hamming	22
4.9.3. Caso 3: bloque equivalente en Fletcher	23
4.10. Simulación estadística	24
4.10.1. Efecto de la probabilidad de error	24
4.10.2. Fallas silenciosas	25
4.10.3. Comportamiento ante una cantidad exacta de errores	26
4.10.4. Overhead	27
4.10.5. Costo computacional	27
5. Discusión	28
5.1. Qué algoritmo tuvo un mejor funcionamiento	28
5.2. Qué algoritmo es más flexible ante tasas de error altas	28
5.3. Cuándo es preferible detectar en lugar de corregir	29
5.4. Es posible engañar al algoritmo	29
5.5. Ventajas y desventajas de cada algoritmo	29
5.6. Dificultad de implementación	30
5.7. Velocidad y costo operativo	30
5.8. Comparación de la redundancia	30
5.9. Relación entre overhead y efectividad	31
5.10. Comportamiento ante cero, uno y múltiples errores	31
6. Comentario sobre el Laboratorio	32
7. Conclusiones	32
8. Citas y referencias	33

1. Resumen

Este informe documenta la implementación y evaluación de dos esquemas de integridad de datos aplicados sobre tramas binarias transmitidas por un canal no confiable. Se construyeron un emisor en C++ y un receptor en Python, cumpliendo la condición de emplear lenguajes distintos a cada lado de la comunicación, y se integraron en ambos un algoritmo de corrección, el Código de Hamming, y uno de detección, el Fletcher checksum con bloques configurables de ocho, dieciséis y treinta y dos bits.

Los programas replican la arquitectura por capas estudiada en clase. La capa de Aplicación solicita el mensaje y el algoritmo, la capa de Presentación traduce entre texto y ASCII binario, la capa de Enlace calcula y verifica la información de redundancia, un módulo de ruido altera bits con probabilidad configurable sobre la totalidad de la trama y la capa de Transmisión traslada el resultado, ya sea de forma manual en pantalla o mediante sockets TCP hacia un receptor que permanece escuchando.

Se evaluaron tres mensajes de cuatro, ocho y sesenta y cuatro bits con ambos algoritmos y bajo tres escenarios de alteración, para un total de dieciocho verificaciones documentadas. A ello se sumó una simulación de Monte Carlo sobre cuatrocientas repeticiones por combinación de parámetros. Los resultados muestran una divergencia marcada entre ambos esquemas: con dos bits alterados, Hamming aceptó erróneamente el 80.50 por ciento de las tramas como si fueran válidas, mientras que Fletcher detectó el 100.00 por ciento de los casos con uno, dos y tres bits alterados. Se construyeron además tres patrones deliberados de alteración que llevan al receptor a aceptar una trama corrupta como intacta, y se identificó la debilidad estructural que permite cada uno.

2. Descripción del Laboratorio

2.1. Planteamiento

El ruido es inherente a toda comunicación. En el modelo de capas, la capa de Enlace ofrece servicios de detección y corrección de errores precisamente porque el medio físico sobre el que viajan los datos no es confiable. El laboratorio busca simular ese mecanismo de protección: el emisor agrega información de redundancia al mensaje, el canal puede alterar cualquier bit de la trama resultante y el receptor debe determinar si lo recibido corresponde a lo enviado.

Las instrucciones del laboratorio presentan dos alcances. Las instrucciones completas

describen el sistema final con arquitectura por capas, codificación ASCII, ruido probabilístico y transmisión real por sockets. La guía resumida indica que en esta fase no habrá conexión real por red y que las pruebas se realizarán trasladando manualmente la salida binaria del emisor al receptor. La implementación entregada resuelve ambos alcances sin sustituir uno por el otro: el modo manual es el flujo principal utilizado en las pruebas documentadas y el modo socket queda disponible como evidencia de la arquitectura completa.

2.2. Objetivos

1. Comprender el funcionamiento de un modelo de capas y los servicios que ofrece cada una.
2. Implementar algoritmos de detección y corrección de errores.
3. Analizar el comportamiento de esos esquemas frente a distintas cantidades de errores.
4. Experimentar con la transmisión de información expuesta a un canal no confiable.
5. Identificar las ventajas y desventajas de cada algoritmo implementado.

2.3. Arquitectura implementada

Ambos programas reproducen la misma secuencia de capas, cada una con la responsabilidad acotada que le corresponde y sin conocimiento de los detalles internos de las demás.

En el emisor, la capa de Aplicación obtiene el mensaje y el algoritmo mediante el servicio de solicitud. La capa de Presentación traduce cada carácter a su representación ASCII binaria de ocho bits cuando la entrada es texto, o bien deja pasar la cadena sin transformarla cuando el usuario ingresa directamente el binario, tal como pide la guía resumida. La capa de Enlace ejecuta el servicio de cálculo de integridad, que invoca el algoritmo elegido y concatena la redundancia al mensaje. El módulo de ruido recorre la trama completa y evalúa cada bit de forma independiente contra la probabilidad configurada, de modo que los bits de paridad y el checksum quedan igualmente expuestos a corrupción. La capa de Transmisión entrega el resultado.

En el receptor, la capa de Transmisión recibe la trama, la capa de Enlace ejecuta el servicio de verificación de integridad, la capa de Presentación decodifica el ASCII

cuando la trama resultó válida o informa la imposibilidad de hacerlo, y la capa de Aplicación comunica el desenlace. El receptor distingue de manera explícita entre tres resultados: trama recibida sin errores, error detectado y corregido con indicación de la posición afectada, y error detectado que obliga a descartar la trama.

2.4. Algoritmos implementados

2.4.1. Código de Hamming

El Código de Hamming es el esquema de corrección. Dado un mensaje de m bits, se determina la cantidad mínima de bits de redundancia r que satisface la desigualdad

$$m + r + 1 \leq 2^r \quad (1)$$

Los bits de paridad se ubican en las posiciones que son potencia de dos y los bits de datos ocupan las posiciones restantes en el mismo orden en que fueron ingresados. Cada bit de paridad cubre las posiciones cuyo índice comparte con él un bit encendido, y su valor se fija con paridad par, es decir, como la suma módulo dos de las posiciones que cubre.

En el receptor se recalculan todas las paridades y se construye el síndrome sumando las posiciones de los bits de control que no se cumplen. Un síndrome igual a cero indica que no se detecta error. Un síndrome comprendido dentro de la longitud de la trama señala directamente la posición del bit invertido, que se corrige antes de extraer el mensaje. Un síndrome que apunta fuera de la trama evidencia un patrón de error superior a la capacidad del código, caso en el que la trama se descarta.

2.4.2. Fletcher checksum

El Fletcher checksum es el esquema de detección. Admite tramas de cualquier longitud y opera con bloques configurables de ocho, dieciséis o treinta y dos bits. Cuando la longitud del mensaje no es múltiplo del tamaño de bloque se aplica relleno de ceros a la derecha, con lo que se preserva el orden de los bits originales.

Sobre el mensaje ya relleno se acumulan dos sumas parciales en aritmética modular. La primera acumula el valor de cada bloque y la segunda acumula los valores sucesivos de la primera, lo que hace al resultado sensible al orden de los bloques:

$$s_1 = (s_1 + b_i) \bmod (2^k - 1) \quad (2)$$

$$s_2 = (s_2 + s_1) \bmod (2^k - 1) \quad (3)$$

donde b_i representa el valor entero del bloque i y k el tamaño de bloque en bits. El checksum resultante ocupa $2k$ bits, con s_2 en la parte alta y s_1 en la parte baja, y se concatena al final del mensaje. El receptor separa esos $2k$ bits finales, recalcula ambas sumas sobre el resto y compara. Al tratarse de un esquema exclusivamente de detección, una discrepancia obliga a descartar la trama sin intentar reconstruir el contenido.

2.5. Herramientas y organización del código

El emisor está escrito en C++ bajo el estándar C++17 y se compiló con g++ 8.1.0 de MinGW-w64 sobre Windows. El receptor está escrito en Python 3.12 y funciona únicamente con la biblioteca estándar. La capa de transmisión del emisor emplea compilación condicional para utilizar Winsock en Windows y sockets POSIX en Linux y macOS, con lo que el mismo código fuente se construye en las tres plataformas sin modificaciones.

El emisor suma 1263 líneas distribuidas en ocho archivos de implementación y siete cabeceras, una por capa y una por algoritmo. El receptor suma 838 líneas distribuidas en paquetes separados para las capas y para los algoritmos. Esta separación permite que ambos algoritmos se integren en la capa de Enlace sin que las capas superiores conozcan su funcionamiento interno.

3. Metodología experimental

3.1. Mensajes de prueba

Se emplearon tres mensajes de longitud claramente distinta, con el fin de observar cómo varía el peso relativo de la redundancia. El tercero corresponde al texto **Hola UVG** codificado en ASCII, lo que permite verificar también el funcionamiento de la capa de Presentación.

Cuadro 1: Mensajes de prueba utilizados en las verificaciones manuales

Etiqueta	Mensaje	Longitud
M1	1101	4 bits
M2	10110011	8 bits
M3	0100100001101111...01000111	64 bits

3.2. Escenarios de alteración

Cada mensaje se procesó con ambos algoritmos y la trama resultante se sometió a los tres escenarios exigidos. En el primero la trama se traslada intacta. En el segundo se invierte manualmente el bit número cinco. En el tercero se invierten los bits cinco y nueve, salvo en las tramas más cortas que nueve bits, donde se emplea la última posición disponible para mantener dos inversiones distintas. La combinación de tres mensajes, dos algoritmos y tres escenarios produce las dieciocho verificaciones documentadas en la sección de resultados.

3.3. Simulación estadística

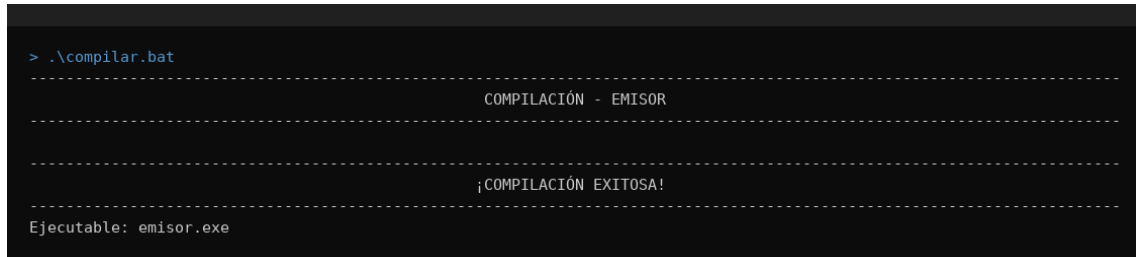
Para respaldar las observaciones puntuales con evidencia agregada se implementó una simulación de Monte Carlo que ejecuta cuatrocientas repeticiones por cada combinación de parámetros, con semilla fija para garantizar la reproducibilidad. Cada ejecución se clasifica en una de tres categorías: entrega correcta cuando el mensaje recuperado coincide con el original, error detectado cuando el receptor rechaza o no puede corregir la trama, y aceptación errónea cuando el receptor entrega un mensaje distinto del original sin advertir el problema. Esta última categoría es la más relevante del análisis, porque corresponde a una falla silenciosa del esquema de integridad.

La simulación varía los cuatro factores solicitados por las instrucciones: el tamaño de las cadenas enviadas, la probabilidad de error del canal, el algoritmo utilizado y el overhead resultante.

4. Resultados

4.1. Compilación y puesta en marcha

El emisor se construye mediante un archivo por lotes en Windows o mediante el Makefile incluido en las demás plataformas. La compilación se realizó con las banderas de advertencia activadas y no produjo errores ni advertencias.



```
> .\compilar.bat
-----
                        COMPILACIÓN - EMISOR
-----
                        ¡COMPILACIÓN EXITOSA!
-----
Ejecutable: emisor.exe
```

Figura 1: Compilación del emisor

4.2. Generación de tramas y overhead

El emisor calcula la trama y reporta el detalle del cálculo realizado, la cantidad de bits de redundancia agregados y el overhead resultante. La figura siguiente corresponde al mensaje M2 procesado con el Código de Hamming, donde se obtiene un código de doce bits totales sobre ocho de datos.

```

> .\emisor

-----
                                UNIVERSIDAD DEL VALLE DE GUATEMALA
-----

Facultad de Ingeniería
Departamento de Computación
Redes
Laboratorio #2
Emisor de Tramas con Detección y Corrección de Errores
André Emilio Pivaral López - 23574
-----

Emisor en C++
Algoritmos: Hamming y Fletcher Checksum
-----

-----
                                MENÚ PRINCIPAL
-----

1. Modo Manual: mostrar la trama en la pantalla
2. Modo Socket: enviar la trama al receptor
3. Salir
Selecciona una Opción: 1
-----

-----
                                CAPA DE APLICACIÓN
-----

1. Ingresar un Mensaje como Binario
2. Ingresar un Mensaje como Texto y Convertirlo a Binario
3. Volver
Selecciona una Opción: 1
Ingresa el Mensaje: 10110011
-----

-----
                                SELECCIÓN DE ALGORITMO
-----

1. Hamming: corrección de errores
2. Fletcher Checksum: detección de errores
3. Volver
Selecciona una Opción: 1
-----

-----
                                CAPA DE PRESENTACIÓN
-----

Tipo de Entrada:      Binaria Directa
Mensaje Binario:     10110011
Bits del Mensaje:     8
-----

-----
                                CAPA DE ENLACE
-----

Algoritmo:            Hamming: corrección de errores
Detalle:              Código (12, 8) con 4 bits de paridad
Bits de Redundancia:  4
Longitud de la Trama: 12
Trama Calculada:      101101100011
-----

-----
                                MÓDULO DE RUIDO
-----

La tasa se expresa como un error por cada N bits transmitidos.
Ingresa 0 para desactivar el ruido y conservar la trama intacta.
Ingresa el Valor de N: 0
Tasa Aplicada:        Ninguna: canal ideal
Bits Alterados:       0
-----

-----
                                CAPA DE TRANSMISIÓN
-----

Modo:                 Traslado Manual
Algoritmo de Destino: Hamming: corrección de errores

Copia la siguiente trama y pégala en el receptor:
101101100011
-----

-----
                                RESUMEN DE LA TRANSMISIÓN
-----

Bits de Datos:        8
Bits de Redundancia:  4
Overhead:              50.00%
Trama Final:          101101100011
-----

-----
                                MENÚ PRINCIPAL
-----

1. Modo Manual: mostrar la trama en la pantalla
2. Modo Socket: enviar la trama al receptor
3. Salir
Selecciona una Opción: 3
-----

-----
                                FIN DEL PROGRAMA EMISOR
-----

```

Figura 2: Generación de la trama del mensaje M2 con el Código de Hamming

La tabla siguiente reúne los resultados obtenidos para los tres mensajes con ambos algoritmos. El overhead se calcula como la proporción de bits adicionales respecto del mensaje original y, en el caso de Fletcher, incluye tanto el checksum como los ceros de relleno.

Cuadro 2: Redundancia y overhead por mensaje y algoritmo

Mensaje	Algoritmo	Bits de datos	Redundancia	Trama	Overhead
M1	Hamming	4	3	7	75.00 %
M1	Fletcher-8	4	20	24	500.00 %
M2	Hamming	8	4	12	50.00 %
M2	Fletcher-8	8	16	24	200.00 %
M3	Hamming	64	7	71	10.94 %
M3	Fletcher-8	64	16	80	25.00 %

La diferencia es estructural. La redundancia de Hamming crece de forma logarítmica respecto de la longitud del mensaje, mientras que la de Fletcher es constante e igual a $2k$ bits más el relleno. En mensajes cortos esto penaliza severamente al checksum, que llega a quintuplicar el tamaño de M1, pero la penalización se diluye rápidamente conforme el mensaje crece.

4.3. Escenario 1: cero errores

La trama se trasladó intacta del emisor al receptor. En los seis casos evaluados el receptor confirmó la integridad de la trama y recuperó el mensaje original sin alteraciones. En Hamming el síndrome calculado resultó igual a cero y en Fletcher el checksum recibido coincidió exactamente con el recalculado.

```

> python receptor.py
-----
                                UNIVERSIDAD DEL VALLE DE GUATEMALA
-----
Facultad de Ingeniería
Departamento de Computación
Redes
Laboratorio #2
Receptor de Tramas con Detección y Corrección de Errores
André Emilio Pivaral López - 23574
-----
Receptor en Python
Algoritmos: Hamming y Fletcher Checksum
-----

                                MENÚ PRINCIPAL
-----
1. Modo Manual: verificar una trama ingresada
2. Modo Socket: escuchar tramas en un puerto
3. Salir
Selecciona una Opción: 1
-----

                                CAPA DE APLICACIÓN
-----
1. Hamming: corrección de errores
2. Fletcher Checksum: detección de errores
3. Volver
Selecciona una Opción: 1
-----

                                CAPA DE TRANSMISIÓN
-----
Pega la trama binaria generada por el emisor.
Puedes alterar manualmente uno o más bits antes de pegarla.
Ingresa la Trama Recibida: 101101100011
-----

                                CAPA DE ENLACE
-----
Algoritmo:                Hamming: corrección de errores
Longitud de la Trama:      12
Trama Recibida:            101101100011
Bits de Datos:             8
Bits de Paridad:           4
Síndrome Calculado:        0
-----

                                ;TRAMA RECIBIDA SIN ERRORES!
-----
Diagnóstico:               Integridad Verificada
Mensaje Recuperado:        10110011
-----

                                CAPA DE PRESENTACIÓN
-----
Decodificación ASCII:      No Aplicada
Motivo:                    El mensaje contiene códigos ASCII no imprimibles
-----

                                CAPA DE APLICACIÓN
-----
Mensaje entregado a la aplicación: la trama llegó íntegra.
-----

                                MENÚ PRINCIPAL
-----
1. Modo Manual: verificar una trama ingresada
2. Modo Socket: escuchar tramas en un puerto
3. Salir
Selecciona una Opción: 3
-----

                                FIN DEL PROGRAMA RECEPTOR
-----

```

Figura 3: Verificación del mensaje M2 con Hamming sin errores

```

> python receptor.py
-----
UNIVERSIDAD DEL VALLE DE GUATEMALA
-----
Facultad de Ingeniería
Departamento de Computación
Redes
Laboratorio #2
Receptor de Tramas con Detección y Corrección de Errores
André Emilio Pivaral López - 23574
-----
Receptor en Python
Algoritmos: Hamming y Fletcher Checksum
-----
-----
MENÚ PRINCIPAL
-----
1. Modo Manual: verificar una trama ingresada
2. Modo Socket: escuchar tramas en un puerto
3. Salir
Selecciona una Opción: 1
-----
CAPA DE APLICACIÓN
-----
1. Hamming: corrección de errores
2. Fletcher Checksum: detección de errores
3. Volver
Selecciona una Opción: 2
-----
TAMAÑO DE BLOQUE
-----
1. 8 Bits
2. 16 Bits
3. 32 Bits
4. Volver
Selecciona una Opción: 1
-----
CAPA DE TRANSMISIÓN
-----
Pega la trama binaria generada por el emisor.
Puedes alterar manualmente uno o más bits antes de pegarla.
Ingresa la Trama Recibida: 101100111011001110110011
-----
CAPA DE ENLACE
-----
Algoritmo: Fletcher Checksum: detección de errores
Longitud de la Trama: 24
Trama Recibida: 101100111011001110110011
Tamaño de Bloque: 8 Bits
Bloques Procesados: 1
Checksum Recibido: 1011001110110011
Checksum Calculado: 1011001110110011
-----
¡TRAMA RECIBIDA SIN ERRORES!
-----
Diagnóstico: Integridad Verificada
Mensaje Recuperado: 10110011
-----
CAPA DE PRESENTACIÓN
-----
Decodificación ASCII: No Aplicada
Motivo: El mensaje contiene códigos ASCII no imprimibles
-----
CAPA DE APLICACIÓN
-----
Mensaje entregado a la aplicación: la trama llegó íntegra.
-----
MENÚ PRINCIPAL
-----
1. Modo Manual: verificar una trama ingresada
2. Modo Socket: escuchar tramas en un puerto
3. Salir
Selecciona una Opción: 3
-----
FIN DEL PROGRAMA RECEPTOR
-----

```

Figura 4: Verificación del mensaje M2 con Fletcher checksum sin errores

4.4. Escenario 2: un error

Se invirtió el bit número cinco de cada trama. El comportamiento de ambos algoritmos difiere de acuerdo con su naturaleza, y esa diferencia constituye el resultado central de esta sección.

El Código de Hamming detectó y corrigió el error en los tres mensajes. El síndrome resultó igual a cinco, señalando directamente la posición alterada, y el receptor restituyó la trama original antes de extraer el mensaje, que coincidió con el emitido.

```

> python receptor.py
-----
UNIVERSIDAD DEL VALLE DE GUATEMALA
-----
Facultad de Ingeniería
Departamento de Computación
Redes
Laboratorio #2
Receptor de Tramas con Detección y Corrección de Errores
André Emilio Pivaral López - 23574
-----
Receptor en Python
Algoritmos: Hamming y Fletcher Checksum
-----

MENÚ PRINCIPAL
-----
1. Modo Manual: verificar una trama ingresada
2. Modo Socket: escuchar tramas en un puerto
3. Salir
Selecciona una Opción: 1
-----

CAPA DE APLICACIÓN
-----
1. Hamming: corrección de errores
2. Fletcher Checksum: detección de errores
3. Volver
Selecciona una Opción: 1
-----

CAPA DE TRANSMISIÓN
-----
Pega la trama binaria generada por el emisor.
Puedes alterar manualmente uno o más bits antes de pegarla.
Ingresa la Trama Recibida: 101111100011
-----

CAPA DE ENLACE
-----
Algoritmo: Hamming: corrección de errores
Longitud de la Trama: 12
Trama Recibida: 101111100011
Bits de Datos: 8
Bits de Paridad: 4
Síndrome Calculado: 5
-----

ERROR DETECTADO Y CORREGIDO
-----
Diagnóstico: Error Simple Corregido
Posición Corregida: Bit número 5
Trama Corregida: 101101100011
Mensaje Recuperado: 10110011
-----
El código de Hamming básico corrige un único bit invertido.
Ante dos o más errores la corrección aplicada puede ser incorrecta.
-----

CAPA DE PRESENTACIÓN
-----
Decodificación ASCII: No Aplicada
Motivo: El mensaje contiene códigos ASCII no imprimibles
-----

CAPA DE APLICACIÓN
-----
Mensaje entregado a la aplicación luego de aplicar la corrección.
-----

MENÚ PRINCIPAL
-----
1. Modo Manual: verificar una trama ingresada
2. Modo Socket: escuchar tramas en un puerto
3. Salir
Selecciona una Opción: 3
-----

FIN DEL PROGRAMA RECEPTOR
-----

```

Figura 5: Corrección de un error simple en el mensaje M2 con Hamming

El Fletcher checksum detectó la discrepancia en los tres mensajes, informó que los checksums no coinciden y descartó la trama sin intentar reconstruirla, que es

exactamente el comportamiento esperado de un algoritmo de detección.

```

> python receptor.py
-----
UNIVERSIDAD DEL VALLE DE GUATEMALA
-----
Facultad de Ingeniería
Departamento de Computación
Redes
Laboratorio #2
Receptor de Tramas con Detección y Corrección de Errores
André Emilio Pivaral López - 23574
-----
Receptor en Python
Algoritmos: Hamming y Fletcher Checksum
-----

MENÚ PRINCIPAL
-----
1. Modo Manual: verificar una trama ingresada
2. Modo Socket: escuchar tramas en un puerto
3. Salir
Selecciona una Opción: 1
-----
CAPA DE APLICACIÓN
-----
1. Hamming: corrección de errores
2. Fletcher Checksum: detección de errores
3. Volver
Selecciona una Opción: 2
-----
TAMAÑO DE BLOQUE
-----
1. 8 Bits
2. 16 Bits
3. 32 Bits
4. Volver
Selecciona una Opción: 1
-----
CAPA DE TRANSMISIÓN
-----
Pega la trama binaria generada por el emisor.
Puedes alterar manualmente uno o más bits antes de pegarla.
Ingresa la Trama Recibida: 101110111011001110110011
-----
CAPA DE ENLACE
-----
Algoritmo: Fletcher Checksum: detección de errores
Longitud de la Trama: 24
Trama Recibida: 101110111011001110110011
Tamaño de Bloque: 8 Bits
Bloques Procesados: 1
Checksum Recibido: 1011001110110011
Checksum Calculado: 1011101110111011
-----
ERROR DETECTADO
-----
Diagnóstico: Los Checksums No Coinciden
Acción: Trama Descartada

Fletcher Checksum es un algoritmo de detección, no de corrección.
La trama se descarta sin intentar reconstruir el mensaje original.
-----
CAPA DE APLICACIÓN
-----
No se entrega mensaje: se notifica el error a la aplicación.
-----
MENÚ PRINCIPAL
-----
1. Modo Manual: verificar una trama ingresada
2. Modo Socket: escuchar tramas en un puerto
3. Salir
Selecciona una Opción: 3
-----
FIN DEL PROGRAMA RECEPTOR
-----

```

Figura 6: Detección de un error simple en el mensaje M2 con Fletcher checksum

En el mensaje M3, que corresponde a texto ASCII, la corrección de Hamming permitió además que la capa de Presentación reconstruyera correctamente la cadena original, lo que evidencia el ascenso completo de la trama por las capas del receptor.

```

> python receptor.py
-----
UNIVERSIDAD DEL VALLE DE GUATEMALA
-----
Facultad de Ingeniería
Departamento de Computación
Redes
Laboratorio #2
Receptor de Tramas con Detección y Corrección de Errores
André Emilio Pivaral López - 23574
-----
Receptor en Python
Algoritmos: Hamming y Fletcher Checksum
-----

MENÚ PRINCIPAL
-----
1. Modo Manual: verificar una trama ingresada
2. Modo Socket: escuchar tramas en un puerto
3. Salir
Selecciona una Opción: 1
-----

CAPA DE APLICACIÓN
-----
1. Hamming: corrección de errores
2. Fletcher Checksum: detección de errores
3. Volver
Selecciona una Opción: 1
-----

CAPA DE TRANSMISIÓN
-----
Pega la trama binaria generada por el emisor.
Puedes alterar manualmente uno o más bits antes de pegarla.
Ingresa la Trama Recibida: 000100001000011101111011000111000010010000001010101010110001000111
-----

CAPA DE ENLACE
-----
Algoritmo: Hamming: corrección de errores
Longitud de la Trama: 71
Trama Recibida: 000100001000011101111011000111000010010000001010101010110001000111
Bits de Datos: 64
Bits de Paridad: 7
Síndrome Calculado: 5
-----

ERROR DETECTADO Y CORREGIDO
-----
Diagnóstico: Error Simple Corregido
Posición Corregida: Bit número 5
Trama Corregida: 000110001000011101111011000111000010010000001010101010110001000111
Mensaje Recuperado: 01001000 01101111 01101100 01100001 00100000 01010101 01010110 01000111
-----
El código de Hamming básico corrige un único bit invertido.
Ante dos o más errores la corrección aplicada puede ser incorrecta.
-----

CAPA DE PRESENTACIÓN
-----
Decodificación ASCII: Aplicada
Texto Recuperado: Hola UVG
-----

CAPA DE APLICACIÓN
-----
Mensaje entregado a la aplicación luego de aplicar la corrección.
-----

MENÚ PRINCIPAL
-----
1. Modo Manual: verificar una trama ingresada
2. Modo Socket: escuchar tramas en un puerto
3. Salir
Selecciona una Opción: 3
-----

FIN DEL PROGRAMA RECEPTOR
-----

```

Figura 7: Corrección de un error simple en el mensaje M3 y recuperación del texto original

4.5. Escenario 3: dos o más errores

Se invirtieron dos bits de cada trama. Aquí aparece la limitación fundamental del Código de Hamming en su forma básica. Con dos inversiones simultáneas el síndrome

resulta igual a la suma exclusiva de ambas posiciones, valor que apunta a una tercera posición no alterada. El receptor, incapaz de distinguir ese caso de un error simple, corrige el bit equivocado e introduce un tercer error, y entrega a la aplicación un mensaje distinto del original sin advertir el problema.

En el mensaje M2 las posiciones cinco y nueve producen un síndrome igual a doce. El receptor corrigió el bit doce y entregó 11111010 en lugar de 10110011.

```

> python receptor.py
-----
UNIVERSIDAD DEL VALLE DE GUATEMALA
-----
Facultad de Ingenieria
Departamento de Computación
Redes
Laboratorio #2
Receptor de Tramas con Detección y Corrección de Errores
André Emilio Pivaral López - 23574
-----
Receptor en Python
Algoritmos: Hamming y Fletcher Checksum
-----

-----
MENÚ PRINCIPAL
-----
1. Modo Manual: verificar una trama ingresada
2. Modo Socket: escuchar tramas en un puerto
3. Salir
Selecciona una Opción: 1
-----

CAPA DE APLICACIÓN
-----
1. Hamming: corrección de errores
2. Fletcher Checksum: detección de errores
3. Volver
Selecciona una Opción: 1
-----

CAPA DE TRANSMISIÓN
-----
Pega la trama binaria generada por el emisor.
Puedes alterar manualmente uno o más bits antes de pegarla.
Ingresa la Trama Recibida: 101111101011
-----

CAPA DE ENLACE
-----
Algoritmo: Hamming: corrección de errores
Longitud de la Trama: 12
Trama Recibida: 101111101011
Bits de Datos: 8
Bits de Paridad: 4
Síndrome Calculado: 12
-----

ERROR DETECTADO Y CORREGIDO
-----
Diagnóstico: Error Simple Corregido
Posición Corregida: Bit número 12
Trama Corregida: 101111101010
Mensaje Recuperado: 11111010
-----
El código de Hamming básico corrige un único bit invertido.
Ante dos o más errores la corrección aplicada puede ser incorrecta.
-----

CAPA DE PRESENTACIÓN
-----
Decodificación ASCII: No Aplicada
Motivo: El mensaje contiene códigos ASCII no imprimibles
-----

CAPA DE APLICACIÓN
-----
Mensaje entregado a la aplicación luego de aplicar la corrección.
-----

MENÚ PRINCIPAL
-----
1. Modo Manual: verificar una trama ingresada
2. Modo Socket: escuchar tramas en un puerto
3. Salir
Selecciona una Opción: 3
-----

FIN DEL PROGRAMA RECEPTOR
-----

```

Figura 8: Corrección incorrecta ante dos errores en el mensaje M2 con Hamming

El Fletcher checksum, en contraste, detectó la alteración en los tres mensajes y descartó la trama. Al no intentar corrección alguna, el algoritmo no puede introducir errores adicionales.

```

> python receptor.py
-----
UNIVERSIDAD DEL VALLE DE GUATEMALA
-----
Facultad de Ingeniería
Departamento de Computación
Redes
Laboratorio #2
Receptor de Tramas con Detección y Corrección de Errores
André Emilio Pivaral López - 23574
-----
Receptor en Python
Algoritmos: Hamming y Fletcher Checksum
-----

MENÚ PRINCIPAL
-----
1. Modo Manual: verificar una trama ingresada
2. Modo Socket: escuchar tramas en un puerto
3. Salir
Selecciona una Opción: 1
-----

CAPA DE APLICACIÓN
-----
1. Hamming: corrección de errores
2. Fletcher Checksum: detección de errores
3. Volver
Selecciona una Opción: 2
-----

TAMAÑO DE BLOQUE
-----
1. 8 Bits
2. 16 Bits
3. 32 Bits
4. Volver
Selecciona una Opción: 1
-----

CAPA DE TRANSMISIÓN
-----
Pega la trama binaria generada por el emisor.
Puedes alterar manualmente uno o más bits antes de pegarla.
Ingresa la Trama Recibida: 101110110011001110110011
-----

CAPA DE ENLACE
-----
Algoritmo: Fletcher Checksum: detección de errores
Longitud de la Trama: 24
Trama Recibida: 101110110011001110110011
Tamaño de Bloque: 8 Bits
Bloques Procesados: 1
Checksum Recibido: 0011001110110011
Checksum Calculado: 1011101110111011
-----

ERROR DETECTADO
-----
Diagnóstico: Los Checksums No Coinciden
Acción: Trama Descartada

Fletcher Checksum es un algoritmo de detección, no de corrección.
La trama se descarta sin intentar reconstruir el mensaje original.
-----

CAPA DE APLICACIÓN
-----
No se entrega mensaje: se notifica el error a la aplicación.
-----

MENÚ PRINCIPAL
-----
1. Modo Manual: verificar una trama ingresada
2. Modo Socket: escuchar tramas en un puerto
3. Salir
Selecciona una Opción: 3
-----

FIN DEL PROGRAMA RECEPTOR
-----

```

Figura 9: Detección de dos errores en el mensaje M2 con Fletcher checksum

La tabla siguiente resume las dieciocho verificaciones realizadas. La columna de entrega válida indica si el mensaje que finalmente recibió la aplicación coincide con el emitido, criterio más estricto que el simple diagnóstico del algoritmo.

Cuadro 3: Resumen de las dieciocho verificaciones manuales

Mensaje	Algoritmo	Escenario	Diagnóstico	Posición	Entrega válida
M1	Hamming	Cero errores	Sin errores	–	Sí
M1	Hamming	Un error	Corregido	5	Sí
M1	Hamming	Dos errores	Corregido	2	No
M1	Fletcher-8	Cero errores	Sin errores	–	Sí
M1	Fletcher-8	Un error	Detectado	–	No
M1	Fletcher-8	Dos errores	Detectado	–	No
M2	Hamming	Cero errores	Sin errores	–	Sí
M2	Hamming	Un error	Corregido	5	Sí
M2	Hamming	Dos errores	Corregido	12	No
M2	Fletcher-8	Cero errores	Sin errores	–	Sí
M2	Fletcher-8	Un error	Detectado	–	No
M2	Fletcher-8	Dos errores	Detectado	–	No
M3	Hamming	Cero errores	Sin errores	–	Sí
M3	Hamming	Un error	Corregido	5	Sí
M3	Hamming	Dos errores	Corregido	12	No
M3	Fletcher-8	Cero errores	Sin errores	–	Sí
M3	Fletcher-8	Un error	Detectado	–	No
M3	Fletcher-8	Dos errores	Detectado	–	No

Conviene señalar la asimetría en la interpretación de la última columna. Cuando Fletcher marca una entrega como no válida lo hace de forma consciente, informando el error y descartando la trama, mientras que en los tres casos de Hamming con dos errores la entrega no válida ocurre sin que el receptor lo advierta.

4.6. Módulo de ruido probabilístico

Además de la alteración manual, se ejercitó el módulo de ruido con una tasa de un error por cada cincuenta bits transmitidos sobre el texto **Hola UVG** codificado con Hamming. El emisor reporta las posiciones que efectivamente resultaron alteradas, lo que permite contrastarlas contra el diagnóstico independiente del receptor.

```

> .\emisor
-----
                                UNIVERSIDAD DEL VALLE DE GUATEMALA
-----
Facultad de Ingeniería
Departamento de Computación
Redes
Laboratorio #2
Emisor de Tramas con Detección y Corrección de Errores
André Emilio Pivaral López - 23574
-----
Emisor en C++
Algoritmos: Hamming y Fletcher Checksum
-----

                                MENÚ PRINCIPAL
-----
1. Modo Manual: mostrar la trama en la pantalla
2. Modo Socket: enviar la trama al receptor
3. Salir
Selecciona una Opción: 1
-----

                                CAPA DE APLICACIÓN
-----
1. Ingresar un Mensaje como Binario
2. Ingresar un Mensaje como Texto y Convertirlo a Binario
3. Volver
Selecciona una Opción: 2
Ingresa el Mensaje: Hola UVG
-----

                                SELECCIÓN DE ALGORITMO
-----
1. Hamming: corrección de errores
2. Fletcher Checksum: detección de errores
3. Volver
Selecciona una Opción: 1
-----

                                CAPA DE PRESENTACIÓN
-----
Texto Original:                Hola UVG
Caracteres:                    8
Mensaje en ASCII Binario:      01001000 01101111 01101100 01100001 00100000 01010101 01010110 01000111
Bits del Mensaje:              64
-----

                                CAPA DE ENLACE
-----
Algoritmo:                     Hamming: corrección de errores
Detalle:                       Código (71, 64) con 7 bits de paridad
Bits de Redundancia:           7
Longitud de la Trama:          71
Trama Calculada:               000110001000011101111011011000111000010010000001010101010110001000111
-----

                                MÓDULO DE RUIDO
-----
La tasa se expresa como un error por cada N bits transmitidos.
Ingresa 0 para desactivar el ruido y conservar la trama intacta.
Ingresa el Valor de N: 50
Tasa Aplicada:                 1/50
Bits Alterados:                0
-----

                                CAPA DE TRANSMISIÓN
-----
Modo:                          Traslado Manual
Algoritmo de Destino:           Hamming: corrección de errores

Copia la siguiente trama y pégala en el receptor:

000110001000011101111011011000111000010010000001010101010110001000111
-----

                                RESUMEN DE LA TRANSMISIÓN
-----
Bits de Datos:                 64
Bits de Redundancia:           7
Overhead:                     10.94%
Trama Final:                   000110001000011101111011011000111000010010000001010101010110001000111
-----

                                MENÚ PRINCIPAL
-----
1. Modo Manual: mostrar la trama en la pantalla
2. Modo Socket: enviar la trama al receptor
3. Salir
Selecciona una Opción: 3
-----

                                FIN DEL PROGRAMA EMISOR
-----

```

Figura 10: Aplicación de ruido probabilístico sobre la trama completa

```

> python receptor.py
-----
UNIVERSIDAD DEL VALLE DE GUATEMALA
-----
Facultad de Ingeniería
Departamento de Computación
Redes
Laboratorio #2
Receptor de Tramas con Detección y Corrección de Errores
André Emilio Pivaral López - 23574
-----
Receptor en Python
Algoritmos: Hamming y Fletcher Checksum
-----

-----
MENÚ PRINCIPAL
-----
1. Modo Manual: verificar una trama ingresada
2. Modo Socket: escuchar tramas en un puerto
3. Salir
Selecciona una Opción: 1
-----

CAPA DE APLICACIÓN
-----
1. Hamming: corrección de errores
2. Fletcher Checksum: detección de errores
3. Volver
Selecciona una Opción: 1
-----

CAPA DE TRANSMISIÓN
-----
Pega la trama binaria generada por el emisor.
Puedes alterar manualmente uno o más bits antes de pegarla.
Ingresa la Trama Recibida: 000110001000011101111011011000111000010010000001010101010110001000111
-----

CAPA DE ENLACE
-----
Algoritmo: Hamming: corrección de errores
Longitud de la Trama: 71
Trama Recibida: 000110001000011101111011011000111000010010000001010101010110001000111
Bits de Datos: 64
Bits de Paridad: 7
Síndrome Calculado: 0
-----

¡TRAMA RECIBIDA SIN ERRORES!
-----
Diagnóstico: Integridad Verificada
Mensaje Recuperado: 01001000 01101111 01101100 01100001 00100000 01010101 01010110 01000111
-----

CAPA DE PRESENTACIÓN
-----
Decodificación ASCII: Aplicada
Texto Recuperado: Hola UVG
-----

CAPA DE APLICACIÓN
-----
Mensaje entregado a la aplicación: la trama llegó íntegra.
-----

MENÚ PRINCIPAL
-----
1. Modo Manual: verificar una trama ingresada
2. Modo Socket: escuchar tramas en un puerto
3. Salir
Selecciona una Opción: 3
-----
FIN DEL PROGRAMA RECEPTOR
-----

```

Figura 11: Verificación de la trama afectada por el canal ruidoso

El módulo evalúa cada bit de forma independiente y actúa sobre la totalidad de la trama, incluidos los bits de paridad, de manera que la propia información de control puede corromperse. Esta condición es relevante porque un error localizado en un bit de redundancia produce un síndrome no nulo aun cuando los datos hayan llegado intactos.

4.7. Transmisión mediante sockets

La capa de Transmisión implementa también el envío real por red. El receptor se coloca a la escucha en un puerto y permanece atendiendo conexiones hasta ser interrumpido, y el emisor establece una conexión TCP y transmite la trama junto con los metadatos que el receptor necesita para verificarla.

```

> ./emisor y > python receptor.py
TERMINAL A: EMISOR

.....
UNIVERSIDAD DEL VALLE DE GUATEMALA
.....
Facultad de Ingeniería
Departamento de Computación
Rodos
Laboratorio #2
Emisor de Tramas con Detección y Corrección de Errores
André Enillo Pivara López - 23374
.....
Emisor en C++
Algoritmo: Hamming y Fletcher Checksum
.....

MENU PRINCIPAL
1. Modo Manual: mostrar la trama en la pantalla
2. Modo Socket: enviar la trama al receptor
3. Salir
Selecciona una Opción: 2

CAPA DE APLICACIÓN
1. Ingresar un Mensaje como Binario
2. Ingresar un Mensaje como Texto y Convertirlo a Binario
3. Volver
Selecciona una Opción: 2
Ingresar el Mensaje: Hola UVG

SELECCIÓN DE ALGORITMO
1. Hamming: corrección de errores
2. Fletcher Checksum: detección de errores
3. Volver
Selecciona una Opción: 1

CAPA DE PRESENTACIÓN
Texto Original:      Hola UVG
Caracteres:          8
Mensaje en ASCII Binario: 01001000 01101111 01101100 01100001 00100000 01010101 01010110 01000111
Bits del Mensaje:     64

CAPA DE ENLACE
Algoritmo:           Hamming: corrección de errores
Detalle:             Código (71, 64) con 7 bits de paridad
Bits de Redundancia: 7
Longitud de la Trama: 71
Trama Calculada:     0001100010000111011101100011100011001000000101010101010001000111

MÓDULO DE RUÍDO
La trama se expresa como un error por cada 8 bits transmitidos.
Ingresar 8 para desactivar el ruido y conservar la trama intacta.
Ingresar el Valor de N: 8
Tipo de Ruido:        Ninguno: canal ideal
Bits Alterados:       0

CAPA DE TRANSMISIÓN
Ingresar el Host del Receptor (127.0.0.1): 127.0.0.1
Ingresar el Puerto del Receptor (50007): 50007

¡TRAMA ENVIADA CORRECTAMENTE!

Destino:              127.0.0.1:50007

RESUMEN DE LA TRANSMISIÓN
Bits de Datos:        64
Bits de Redundancia:  7
Overhead:             10.94%
Trama Final:          0001100010000111011101100011100011001000000101010101010001000111

MENU PRINCIPAL
1. Modo Manual: mostrar la trama en la pantalla
2. Modo Socket: enviar la trama al receptor
3. Salir
Selecciona una Opción: 3

FIN DEL PROGRAMA EMISOR

TERMINAL B: RECEPTOR

.....
UNIVERSIDAD DEL VALLE DE GUATEMALA
.....
Facultad de Ingeniería
Departamento de Computación
Rodos
Laboratorio #2
Receptor de Tramas con Detección y Corrección de Errores
André Enillo Pivara López - 23374
.....
Receptor en Python
Algoritmo: Hamming y Fletcher Checksum
.....

MENU PRINCIPAL
1. Modo Manual: verificar una trama ingresada
2. Modo Socket: escuchar tramas en un puerto
3. Salir
Selecciona una Opción: 2

CAPA DE TRANSMISIÓN
El receptor quedará a la espera de las tramas enviadas por el emisor.
Presione Ctrl+C para detener la escucha y volver al menú principal.
Ingresar el Puerto de Escucha (50007): 50007

¡RECEPTOR EN ESCUCHA!

Puerto:              50007

TRAMA ENTRANTE
Origen:               127.0.0.1:50007
Algoritmo Declarado:  HAMMING

CAPA DE ENLACE
Algoritmo:           Hamming: corrección de errores
Longitud de la Trama: 71
Trama Recibida:      0001100010000111011101100011100011001000000101010101010001000111
Bits de Datos:        64
Bits de Paridad:      7
Síndrome Calculado:   0

¡TRAMA RECIBIDA SIN ERRORES!

Diagnóstico:          Integridad Verificada
Mensaje Recuperado:   01001000 01101111 01101100 01100001 00100000 01010101 01010110 01000111

CAPA DE PRESENTACIÓN
Decodificación ASCII: Aplicada
Texto Recuperado:     Hola UVG

```

Figura 12: Transmisión y recepción mediante sockets sobre el puerto 50007

4.8. Tamaño de bloque configurable

Las instrucciones exigen que las tres opciones de tamaño de bloque del Fletcher checksum sean configurables. La figura siguiente corresponde al mensaje M2 procesado con bloques de treinta y dos bits, caso en el que el relleno alcanza veinticuatro ceros y el checksum ocupa sesenta y cuatro bits.

```

> python receptor.py
-----
                                UNIVERSIDAD DEL VALLE DE GUATEMALA
-----
Facultad de Ingeniería
Departamento de Computación
Redes
Laboratorio #2
Receptor de Tramas con Detección y Corrección de Errores
André Emilio Pivaral López - 23574
-----
Receptor en Python
Algoritmos: Hamming y Fletcher Checksum
-----

                                MENÚ PRINCIPAL
-----
1. Modo Manual: verificar una trama ingresada
2. Modo Socket: escuchar tramas en un puerto
3. Salir
Selecciona una Opción: 1
-----

                                CAPA DE APLICACIÓN
-----
1. Hamming: corrección de errores
2. Fletcher Checksum: detección de errores
3. Volver
Selecciona una Opción: 2
-----

                                TAMAÑO DE BLOQUE
-----
1. 8 Bits
2. 16 Bits
3. 32 Bits
4. Volver
Selecciona una Opción: 3
-----

                                CAPA DE TRANSMISIÓN
-----
Pega la trama binaria generada por el emisor.
Puedes alterar manualmente uno o más bits antes de pegarla.
Ingresa la Trama Recibida: 10110011000000000000000000000000101100110000000000000000000000001011001100000000000000000000000
-----

                                CAPA DE ENLACE
-----
Algoritmo: Fletcher Checksum: detección de errores
Longitud de la Trama: 96
Trama Recibida: 10110011000000000000000000000000101100110000000000000000000000001011001100000000000000000000000
Tamaño de Bloque: 32 Bits
Bloques Procesados: 1
Checksum Recibido: 1011001100000000000000000000000010110011000000000000000000000000
Checksum Calculado: 1011001100000000000000000000000010110011000000000000000000000000
-----

                                ¡TRAMA RECIBIDA SIN ERRORES!
-----
Diagnóstico: Integridad Verificada
Mensaje Recuperado: 10110011 00000000 00000000 00000000
-----

                                CAPA DE PRESENTACIÓN
-----
Decodificación ASCII: No Aplicada
Motivo: El mensaje contiene códigos ASCII no imprimibles
-----

                                CAPA DE APLICACIÓN
-----
Mensaje entregado a la aplicación: la trama llegó integra.
-----

                                MENÚ PRINCIPAL
-----
1. Modo Manual: verificar una trama ingresada
2. Modo Socket: escuchar tramas en un puerto
3. Salir
Selecciona una Opción: 3
-----

                                FIN DEL PROGRAMA RECEPTOR
-----

```

Figura 13: Verificación con bloques de treinta y dos bits

4.9. Intento de engañar a los algoritmos

Se construyeron de forma deliberada patrones de alteración que llevan al receptor a concluir que una trama corrupta está intacta. Los tres casos encontrados se verifican automáticamente en cada ejecución contra las implementaciones reales del receptor.

4.9.1. Caso 1: síndrome nulo en Hamming

Al invertir los bits en las posiciones tres, cinco y seis de una trama de Hamming, el síndrome resulta igual a cero porque la suma exclusiva de esas posiciones se anula. Todas las paridades vuelven a cumplirse y el receptor concluye que la trama está intacta pese a contener tres errores, entregando un mensaje completamente distinto del emitido.

```
> python demostracion_engano.py
-----
                        CASO 1: HAMMING CON SÍNDROME NULO
-----
Mensaje Original:      10110011
Trama Emitida:         101101100011
Bits Invertidos:       3, 5, 6
Trama Alterada:        100110100011
Síndrome Calculado:    0
Diagnóstico del Receptor: Sin Errores
Mensaje Entregado:     01010011
-----
                        ¡ENGAÑO LOGRADO!
-----
El síndrome es la suma XOR de las posiciones alteradas. Cuando ese
valor se anula, todas las paridades vuelven a cumplirse y el receptor
concluye que la trama está intacta pese a contener tres errores.
```

Figura 14: Patrón de tres bits que anula el síndrome de Hamming

4.9.2. Caso 2: corrección equivocada en Hamming

Con dos bits invertidos el síndrome apunta a una tercera posición y el receptor aplica una corrección incorrecta. El resultado no solo conserva los dos errores originales sino que agrega uno más, y el diagnóstico informa una corrección exitosa que en realidad no lo fue.

```

> python demostracion_engano.py
-----
                                CASO 2: HAMMING CON DOS ERRORES
-----
Mensaje Original:      10110011
Trama Emitida:         101101100011
Bits Invertidos:       5, 9
Trama Alterada:        101111101011
Síndrome Calculado:    12
Diagnóstico del Receptor: Error Detectado y Corregido
Posición Corregida:    Bit número 12
Mensaje Entregado:     11111010
-----
                                ¡ENGAÑO LOGRADO!
-----
La distancia mínima del código de Hamming básico es tres, lo que
permite corregir un solo bit. Con dos errores el síndrome apunta a una
tercera posición y el receptor introduce un error adicional al corregir.

```

Figura 15: Corrección incorrecta de Hamming ante dos bits invertidos

4.9.3. Caso 3: bloque equivalente en Fletcher

El Fletcher checksum opera en aritmética módulo $2^k - 1$, por lo que el valor cero y el valor $2^k - 1$ son congruentes. Sustituir un bloque completamente nulo por un bloque de unos deja ambas sumas parciales sin cambio y el checksum coincide, de modo que una alteración de ocho bits consecutivos pasa completamente inadvertida.

```

> python demostracion_engano.py
-----
                                CASO 3: FLETCHER CON BLOQUE EQUIVALENTE
-----
Mensaje Original:      000000001001101
Trama Emitida:         00000000100110101001101001101
Bits Invertidos:       Los 8 bits del primer bloque
Trama Alterada:        1111111010011010100110101001101
Checksum Recibido:     01001101001101
Checksum Calculado:    01001101001101
Diagnóstico del Receptor: Sin Errores
-----
                                ¡ENGAÑO LOGRADO!
-----
El algoritmo trabaja en módulo  $2^k$  menos uno, por lo que el valor cero
y el valor  $2^k$  menos uno son congruentes. Sustituir un bloque nulo por
un bloque de unos deja sum1 y sum2 sin cambio y el checksum coincide.

```

Figura 16: Equivalencia entre un bloque nulo y un bloque de unos en Fletcher

Los tres patrones son deliberados y no implican que los algoritmos fallen con frecuencia ante ruido aleatorio. Lo que demuestran es que existen alteraciones específicas que ninguno de los dos esquemas puede distinguir de una trama intacta, y que la ausencia de un diagnóstico de error no equivale a una garantía de integridad.

4.10. Simulación estadística

4.10.1. Efecto de la probabilidad de error

Se evaluaron siete probabilidades de error por bit sobre mensajes de sesenta y cuatro bits. La figura siguiente muestra la proporción de mensajes entregados correctamente por cada algoritmo.

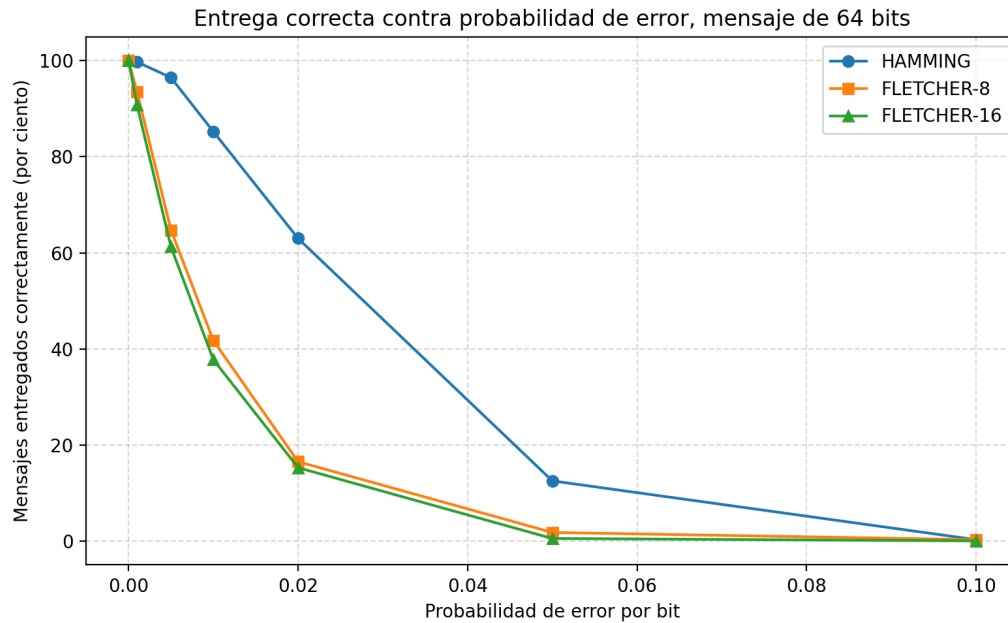


Figura 17: Entrega correcta contra probabilidad de error del canal

Cuadro 4: Entrega correcta según la probabilidad de error, en porcentaje

Probabilidad	Hamming	Fletcher-8	Fletcher-16
0.000	100.00	100.00	100.00
0.001	99.75	93.50	90.75
0.005	96.50	64.75	61.25
0.010	85.25	41.75	37.75
0.020	63.00	16.50	15.25
0.050	12.50	1.75	0.50
0.100	0.25	0.25	0.00

Leída de forma aislada, esta tabla parecería favorecer ampliamente a Hamming. La ventaja es real y proviene de su capacidad de corrección: mientras el canal produzca a lo sumo un error por trama, Hamming lo repara y entrega el mensaje, en tanto que Fletcher se ve obligado a descartar la trama completa. Sin embargo, la lectura cambia

por completo al observar cómo se distribuyen los casos que no fueron entregados correctamente.

4.10.2. Fallas silenciosas

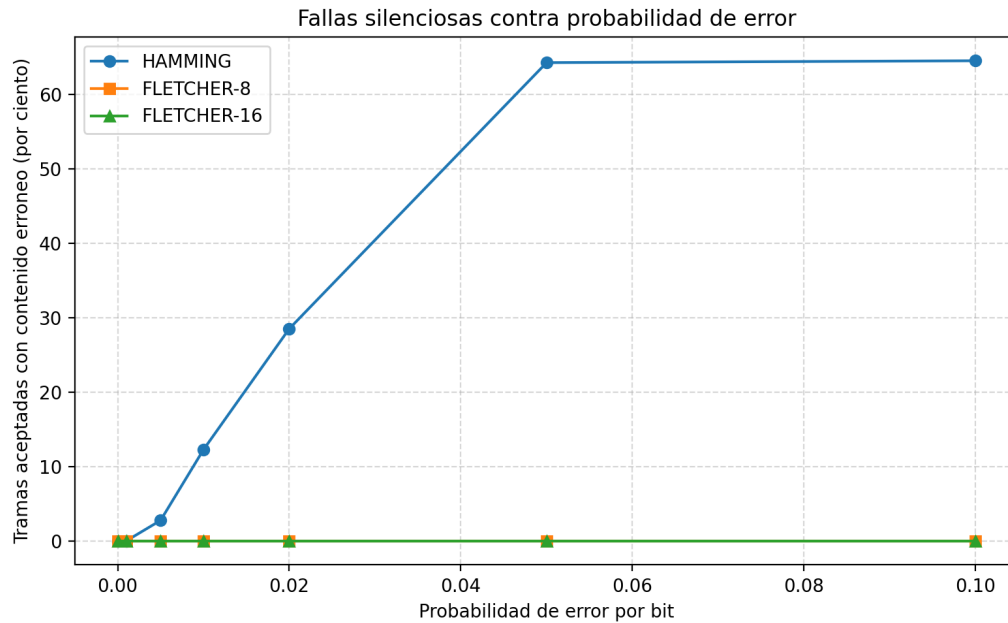


Figura 18: Tramas aceptadas con contenido erróneo contra probabilidad de error

Cuadro 5: Aceptación errónea según la probabilidad de error, en porcentaje

Probabilidad	Hamming	Fletcher-8	Fletcher-16
0.000	0.00	0.00	0.00
0.001	0.00	0.00	0.00
0.005	2.75	0.00	0.00
0.010	12.25	0.00	0.00
0.020	28.50	0.00	0.00
0.050	64.25	0.00	0.00
0.100	64.50	0.00	0.00

El contraste es categórico. A partir de una probabilidad de error de 0.005 el Código de Hamming comienza a entregar mensajes corruptos sin advertirlo, y con una tasa de 0.05 lo hace en el 64.25 por ciento de los casos. El Fletcher checksum no registró ninguna aceptación errónea en ninguna de las configuraciones evaluadas.

4.10.3. Comportamiento ante una cantidad exacta de errores

Para aislar el efecto de la cantidad de errores del efecto de la longitud de la trama, se repitió el experimento invirtiendo un número exacto de bits en posiciones distintas escogidas al azar.

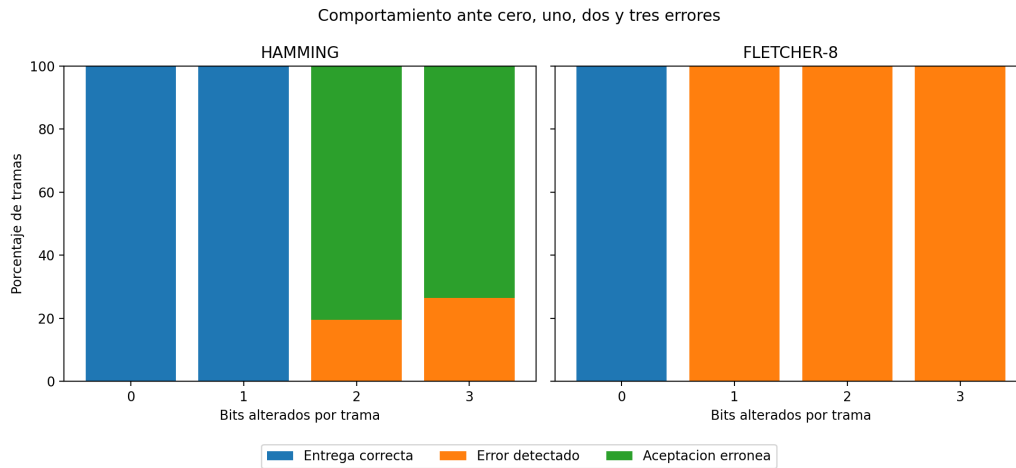


Figura 19: Comportamiento ante cero, uno, dos y tres errores

Cuadro 6: Distribución de resultados según la cantidad exacta de bits alterados, en porcentaje

Algoritmo	Bits alterados	Entrega correcta	Error detectado	Aceptación errónea
Hamming	0	100.00	0.00	0.00
Hamming	1	100.00	0.00	0.00
Hamming	2	0.00	19.50	80.50
Hamming	3	0.00	26.50	73.50
Fletcher-8	0	100.00	0.00	0.00
Fletcher-8	1	0.00	100.00	0.00
Fletcher-8	2	0.00	100.00	0.00
Fletcher-8	3	0.00	100.00	0.00

Hamming corrigió el 100.00 por ciento de los errores simples, resultado coherente con su distancia mínima de tres. Con dos bits alterados aceptó erróneamente el 80.50 por ciento de las tramas y con tres el 73.50 por ciento. Fletcher detectó el 100.00 por ciento de los casos con uno, dos y tres bits alterados dentro de la muestra evaluada, aunque este resultado no debe interpretarse como una garantía absoluta, pues el caso construido en la sección anterior demuestra que existen patrones específicos que el algoritmo no distingue.

4.10.4. Overhead

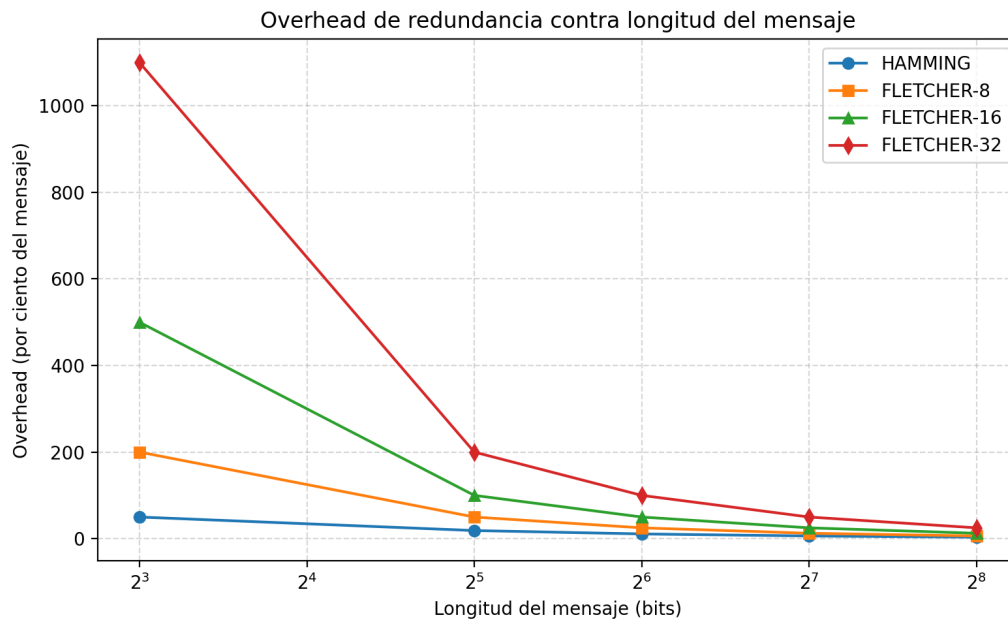


Figura 20: Overhead de redundancia contra longitud del mensaje

Cuadro 7: Overhead porcentual según la longitud del mensaje

Longitud	Hamming	Fletcher-8	Fletcher-16	Fletcher-32
8 bits	50.00	200.00	500.00	1100.00
32 bits	18.75	50.00	100.00	200.00
64 bits	10.94	25.00	50.00	100.00
128 bits	6.25	12.50	25.00	50.00
256 bits	3.52	6.25	12.50	25.00

El overhead de Hamming decrece de forma logarítmica y el de Fletcher de forma hiperbólica, ya que su redundancia es constante. En mensajes muy cortos ambos esquemas resultan desproporcionados, con un extremo de 1100 por ciento para bloques de treinta y dos bits sobre un mensaje de ocho, situación en la que el checksum es once veces más grande que el dato que protege.

4.10.5. Costo computacional

Se midió el tiempo promedio de codificación y verificación sobre dos mil mensajes de sesenta y cuatro bits en la misma implementación de referencia en Python. El Código de Hamming requirió 70.8 microsegundos por mensaje frente a 6.8 microsegundos

del Fletcher checksum, una relación de aproximadamente diez a uno. La diferencia se explica por la naturaleza de cada cálculo: Hamming recorre la trama completa una vez por cada bit de paridad, con un costo que crece como el producto de la longitud por el logaritmo de la longitud, mientras que Fletcher realiza dos sumas modulares por bloque en un único recorrido lineal.

5. Discusión

5.1. Qué algoritmo tuvo un mejor funcionamiento

La respuesta depende del criterio de evaluación, y esa ambigüedad es en sí misma el hallazgo más interesante del laboratorio. Si el criterio es la proporción de mensajes entregados correctamente, el Código de Hamming es superior en todo el rango evaluado, con 85.25 por ciento frente a 41.75 por ciento del checksum a una probabilidad de error de 0.01. Si el criterio es la confiabilidad del diagnóstico, es decir, la capacidad de no afirmar que una trama es válida cuando no lo es, el Fletcher checksum es categóricamente superior, con cero aceptaciones erróneas frente a un 12.25 por ciento de Hamming en esa misma configuración.

Para un sistema donde entregar un dato corrupto sin advertirlo es peor que no entregar nada, como una transferencia bancaria o la actualización de un firmware, Fletcher es la elección correcta. Para un sistema donde la retransmisión es costosa o imposible y una degradación ocasional es tolerable, como una transmisión de voz o un enlace satelital de un solo sentido, Hamming es preferible.

5.2. Qué algoritmo es más flexible ante tasas de error altas

El Fletcher checksum. Su capacidad de detección se mantuvo estable en todo el rango evaluado, alcanzando el 99.75 por ciento de detección a una probabilidad de 0.1, mientras que su tasa de aceptación errónea permaneció en cero. Hamming, en cambio, se degrada de forma acelerada: su capacidad de corrección está diseñada para un solo bit por trama y a partir de una probabilidad de 0.02 la mayoría de las tramas de sesenta y cuatro bits contienen dos o más errores, condición que lo lleva al régimen de corrección incorrecta.

Debe matizarse que la flexibilidad de Fletcher es de detección, no de entrega. A tasas altas detecta casi todo pero entrega casi nada, lo que en un sistema real se traduciría en retransmisiones continuas.

5.3. Cuándo es preferible detectar en lugar de corregir

La detección es preferible cuando existe un canal de retorno que permita solicitar la retransmisión y cuando el costo de un dato corrupto no advertido supera el costo de reenviar. Es el caso de la mayoría de las redes de datos actuales, donde los protocolos superiores implementan confirmaciones y retransmisiones.

La corrección es preferible cuando no existe canal de retorno, cuando la latencia de la retransmisión es inaceptable o cuando el emisor no está disponible para reenviar. El almacenamiento en memoria, las comunicaciones espaciales y la difusión en un solo sentido son ejemplos típicos.

Un factor adicional es el overhead. La corrección exige conocer la posición del error, no solo su existencia, lo que requiere estructuralmente más información. En este laboratorio esa relación queda parcialmente enmascarada porque Fletcher agrega bits fijos que resultan onerosos en mensajes cortos, pero la tendencia se invierte al crecer el mensaje.

5.4. Es posible engañar al algoritmo

Sí, y se documentaron tres casos concretos. En Hamming, cualquier patrón de tres o más inversiones cuyas posiciones tengan suma exclusiva igual a cero produce un síndrome nulo y el receptor declara la trama intacta. En Hamming, dos inversiones simultáneas producen un síndrome que apunta a una tercera posición y provocan una corrección incorrecta reportada como exitosa. En Fletcher, sustituir un bloque nulo por un bloque de unos no altera ninguna de las dos sumas parciales, porque cero y $2^k - 1$ son congruentes en el módulo empleado.

La causa común de las dos primeras debilidades es la distancia mínima del código, igual a tres, que permite corregir un error o detectar dos, pero no ambas cosas simultáneamente ni nada más allá. La causa de la tercera es la elección del módulo del algoritmo, que introduce una clase de equivalencia entre dos valores de bloque distintos.

5.5. Ventajas y desventajas de cada algoritmo

El Código de Hamming tiene como ventajas la corrección efectiva de errores simples sin retransmisión, un overhead que crece logarítmicamente y la identificación exacta de la posición afectada. Como desventajas presenta una capacidad limitada a un

solo bit, la corrupción activa del mensaje ante dos errores y la ausencia de cualquier señal de alarma en ese caso.

El Fletcher checksum tiene como ventajas una detección robusta e independiente de la cantidad de errores, un cálculo lineal y económico, tamaño de bloque configurable y la imposibilidad de introducir errores nuevos. Como desventajas presenta la ausencia total de corrección, un overhead constante que resulta desproporcionado en mensajes cortos y la clase de equivalencia derivada de su aritmética modular.

5.6. Dificultad de implementación

El Fletcher checksum resultó considerablemente más sencillo. Su lógica se reduce a dividir la trama en bloques, acumular dos sumas modulares y comparar el resultado, sin manipulación de posiciones individuales. La única decisión de diseño relevante fue fijar la convención de relleno y el orden de concatenación de las dos sumas, que debe ser idéntica en ambos lenguajes.

El Código de Hamming exigió mayor cuidado. La numeración de posiciones inicia en uno y no en cero, la ubicación de los bits de paridad depende de las potencias de dos y la cobertura de cada bit de control se determina mediante una operación sobre los índices. Cualquier discrepancia entre las convenciones del emisor y del receptor produce tramas incompatibles. Para garantizar la equivalencia entre la implementación en C++ y la de Python se verificó que ambas produjeran cadenas idénticas sobre trescientos mensajes aleatorios de longitud variable.

5.7. Velocidad y costo operativo

La medición sobre dos mil mensajes de sesenta y cuatro bits arrojó 70.8 microsegundos por mensaje para Hamming frente a 6.8 microsegundos para Fletcher, una relación cercana a diez a uno. La diferencia responde a la estructura de cada algoritmo. Fletcher realiza un único recorrido lineal con dos sumas modulares por bloque, mientras que Hamming recorre toda la trama una vez por cada bit de paridad, con un costo proporcional al producto de la longitud por su logaritmo. En un enlace de alta velocidad esa diferencia se vuelve determinante.

5.8. Comparación de la redundancia

Para un mensaje de sesenta y cuatro bits, Hamming agrega siete bits y Fletcher con bloques de ocho agrega dieciséis, más el relleno que corresponda. La diferencia se

acentúa en mensajes cortos, donde Fletcher con bloques de treinta y dos bits llega a un overhead de 1100 por ciento sobre un mensaje de ocho bits, y se atenúa en mensajes largos, donde ambos convergen hacia valores de un dígito.

Un aspecto operativo importante es que el tamaño de bloque de Fletcher debe elegirse en función de la longitud esperada del mensaje. Un bloque de treinta y dos bits sobre mensajes cortos desperdicia una cantidad considerable de ancho de banda sin aportar detección adicional proporcional.

5.9. Relación entre overhead y efectividad

Los datos obtenidos no respaldan una relación directa entre mayor overhead y mayor efectividad. Fletcher-16 duplica la redundancia de Fletcher-8 y sin embargo no mejora la detección en la muestra evaluada, donde ambos alcanzan cero aceptaciones erróneas y una entrega correcta ligeramente inferior en el caso de dieciséis bits, precisamente por su trama más larga y por tanto más expuesta al ruido.

Más ilustrativo aún es el caso de Hamming, que con un overhead de 10.94 por ciento sobre sesenta y cuatro bits produce el 80.50 por ciento de aceptaciones erróneas ante dos errores, mientras que Fletcher-8, con 25.00 por ciento de overhead, no produce ninguna. Lo determinante no es cuánta redundancia se agrega sino qué propiedad matemática se obtiene a cambio. La conclusión práctica es que el overhead debe evaluarse junto con el modelo de errores esperado del canal y no como una medida de calidad en sí misma.

5.10. Comportamiento ante cero, uno y múltiples errores

Ante cero errores ambos algoritmos se comportaron de forma idéntica y correcta en la totalidad de los casos. Ante un error las diferencias son de diseño y no de calidad: Hamming corrigió el 100.00 por ciento y Fletcher detectó el 100.00 por ciento, cada uno cumpliendo exactamente su función. Ante dos o más errores las trayectorias divergen por completo, con Hamming aceptando erróneamente el 80.50 por ciento de las tramas y Fletcher manteniendo una detección total dentro de la muestra evaluada.

La transición de uno a dos errores marca la frontera de capacidad del código de Hamming. No se trata de una degradación gradual sino de un cambio de régimen: el algoritmo pasa de corregir siempre a corregir mal casi siempre, sin ningún estado intermedio de advertencia.

6. Comentario sobre el Laboratorio

El laboratorio resultó especialmente valioso porque obliga a confrontar una intuición equivocada bastante común, la de suponer que un algoritmo de corrección es simplemente una versión mejorada de uno de detección. La evidencia experimental muestra lo contrario: son herramientas con propósitos distintos y con modos de falla opuestos. Un algoritmo de detección falla no entregando información, mientras que uno de corrección puede fallar entregando información equivocada con apariencia de correcta, que es el modo de falla más peligroso en un sistema real.

El requisito de implementar el emisor y el receptor en lenguajes distintos aportó un valor pedagógico que no resulta evidente al leer las instrucciones. Obliga a definir con precisión el formato de la trama, el orden de los bits, la convención de relleno y la numeración de las posiciones, porque no existe la posibilidad de compartir estructuras de datos entre ambos lados. Es exactamente el problema que enfrenta cualquier protocolo real y una fuente frecuente de errores de interoperabilidad.

La construcción deliberada de casos que engañan a los algoritmos fue la parte más instructiva del trabajo. Pasar de conocer teóricamente la distancia mínima de un código a construir un patrón concreto de tres bits que la explota consolida el concepto de una manera que la sola lectura no logra.

Como dificultad principal se encuentra la coordinación de convenciones entre ambas implementaciones. Una diferencia en el orden de concatenación de las sumas parciales de Fletcher o en la numeración de las posiciones de Hamming produce tramas mutuamente incomprensibles sin ningún mensaje de error evidente. La verificación cruzada automatizada entre ambos lenguajes fue indispensable para detectar y descartar ese tipo de discrepancias.

7. Conclusiones

1. El Código de Hamming corrigió el 100.00 por ciento de los errores simples en las dieciocho verificaciones manuales y en la simulación de cuatrocientas repeticiones, confirmando que un código con distancia mínima de tres corrige de manera confiable un bit invertido por trama.
2. El Código de Hamming en su forma básica no solo pierde su capacidad ante dos errores sino que corrompe activamente el mensaje, aceptando erróneamente el 80.50 por ciento de las tramas y entregando datos incorrectos sin ninguna señal de advertencia. Este modo de falla es más peligroso que la simple incapacidad

de corregir.

3. El Fletcher checksum detectó el 100.00 por ciento de los casos con uno, dos y tres bits alterados y no registró ninguna aceptación errónea en ninguna de las configuraciones evaluadas, lo que lo convierte en la opción adecuada cuando la integridad del diagnóstico es prioritaria sobre la disponibilidad del dato.
4. Ambos algoritmos pueden ser engañados mediante patrones específicos de alteración. La suma exclusiva nula de posiciones en Hamming y la congruencia entre el bloque nulo y el bloque de unos en Fletcher son debilidades estructurales derivadas del diseño matemático de cada esquema, no defectos de la implementación.
5. El overhead no predice la efectividad. Fletcher-8, con un 25.00 por ciento de redundancia sobre mensajes de sesenta y cuatro bits, superó a Fletcher-16, con un 50.00 por ciento, en entrega correcta sin perder detección, y Hamming, con un 10.94 por ciento, produjo el mayor número de fallas silenciosas del estudio.
6. La elección del algoritmo debe responder al modelo de errores del canal y al costo relativo de cada tipo de falla, no a una noción abstracta de superioridad. Con un canal de buena calidad y sin retorno, la corrección aporta valor; con un canal ruidoso o cuando un dato corrupto es inaceptable, la detección con retransmisión es la estrategia correcta.
7. La implementación por capas demostró su utilidad práctica al permitir que ambos algoritmos se integraran en la capa de Enlace sin modificar las capas superiores, y que el traslado manual y la transmisión por sockets coexistieran intercambiando únicamente la capa de Transmisión.

8. Citas y referencias

1. Hamming, R. W. (1950). Error detecting and error correcting codes. *The Bell System Technical Journal*, 29(2), 147–160.
2. Fletcher, J. G. (1982). An arithmetic checksum for serial transmissions. *IEEE Transactions on Communications*, 30(1), 247–252.
3. Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer Networks* (5.^a ed.). Prentice Hall.
4. Stallings, W. (2013). *Data and Computer Communications* (10.^a ed.). Pearson.

5. Zweig, J., & Partridge, C. (1990). *TCP alternate checksum options* (RFC 1146). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc1146>
6. Maxino, T. C., & Koopman, P. J. (2009). The effectiveness of checksums for embedded control networks. *IEEE Transactions on Dependable and Secure Computing*, 6(1), 59–72.
7. Lin, S., & Costello, D. J. (2004). *Error Control Coding* (2.^a ed.). Prentice Hall.